

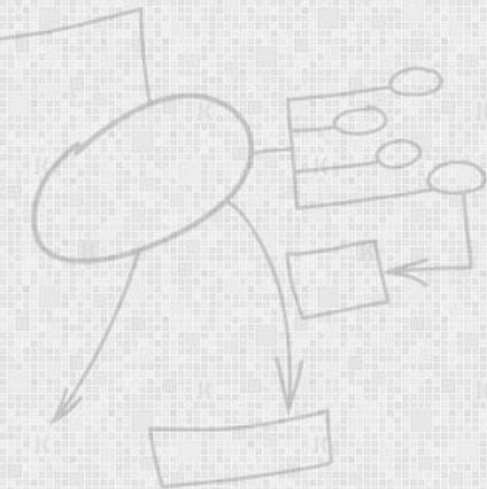
EC-360

SOFTWARE ENGINEERING

Lecture 5

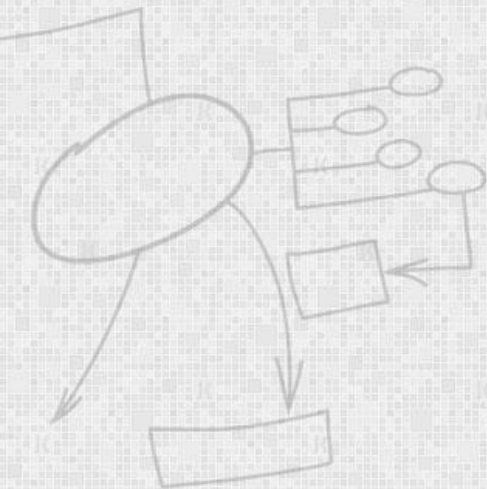
Software Architecture

By
Dr Wasi Haider Butt



Today's Lecture: Agenda

- Architecture
- Architectural Styles
- Distributed Architectures



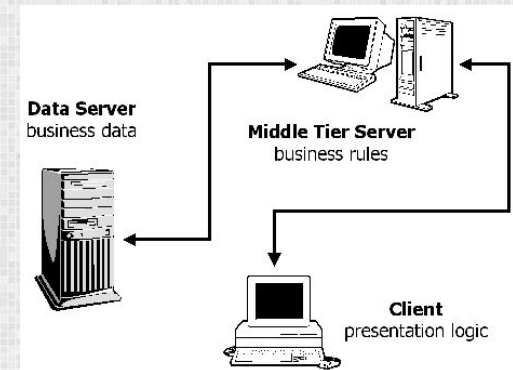
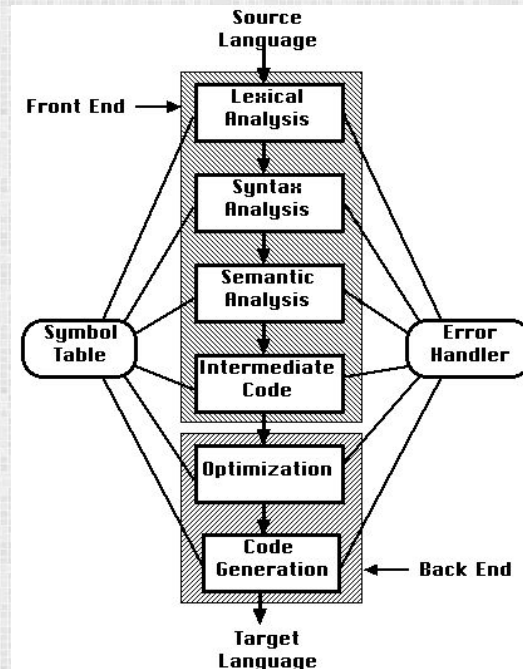
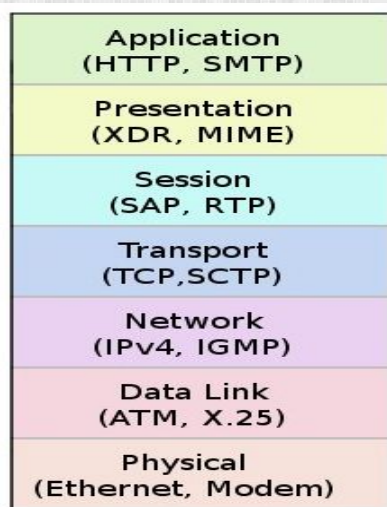
Software Architecture

- System viewed as collection of subsystems and/or interaction in terms of communication and control
- The design process for identifying the sub-systems making up a system and the **framework** for sub-system control and communication
- Structure Vs Behaviour



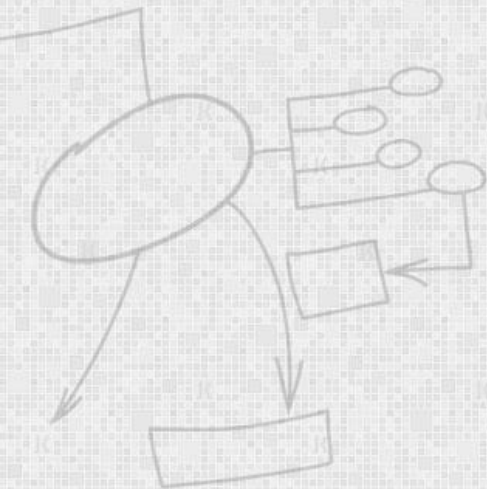
What Is Software Architecture?

- Framework, mechanism, way, shape, form, means, organization, arrangement
- **The Big Picture!**



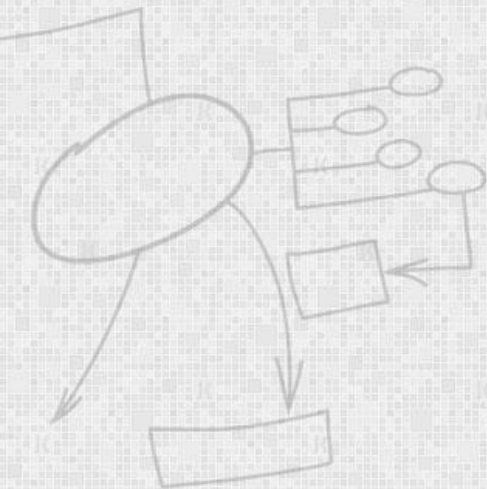
Elements of Software Architecture

- Components
- Connectors
- Configuration topologies



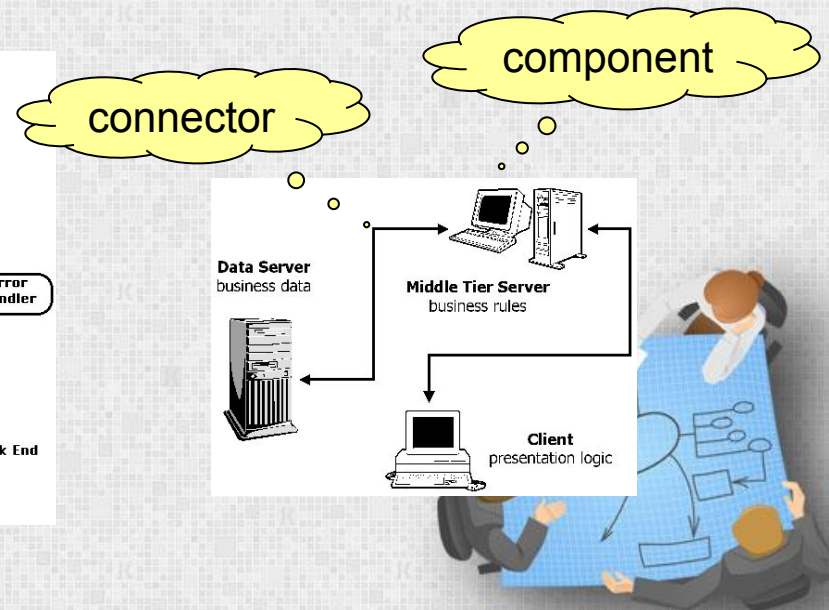
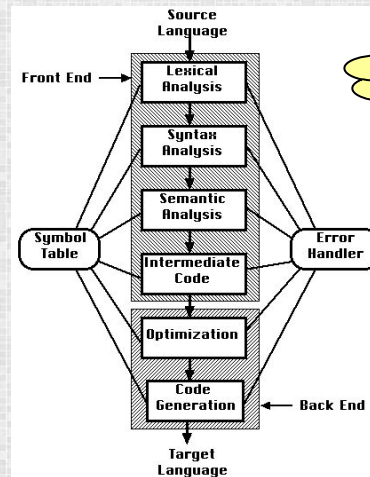
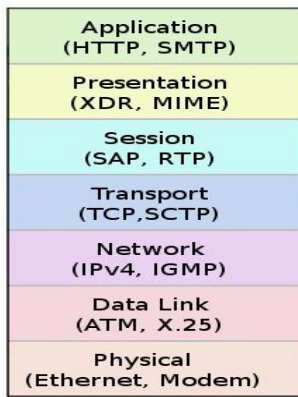
Software Components

- Elements that encapsulate processing and data in a system's architecture
- Often represented as boxes in diagrams.



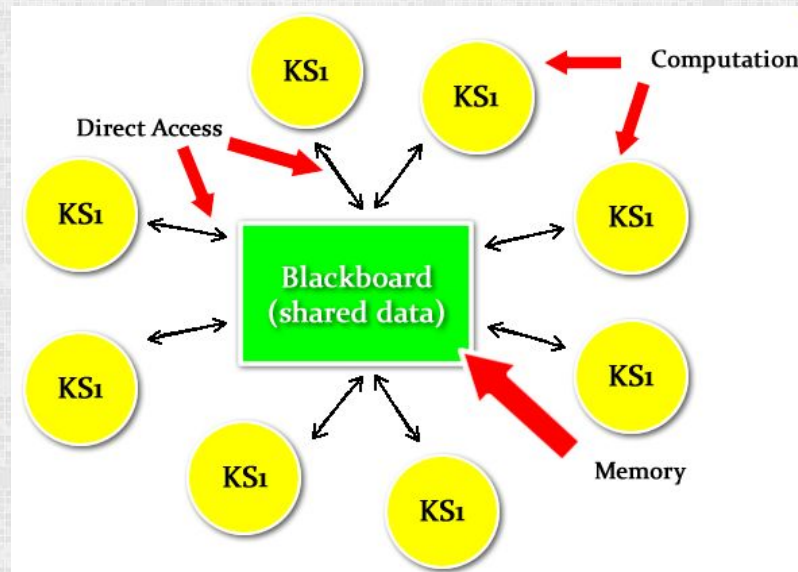
Connectors

- Elements that are tasked with effecting and regulating **interactions** among components.
- Application independent.
- Often represented as lines in diagrams.

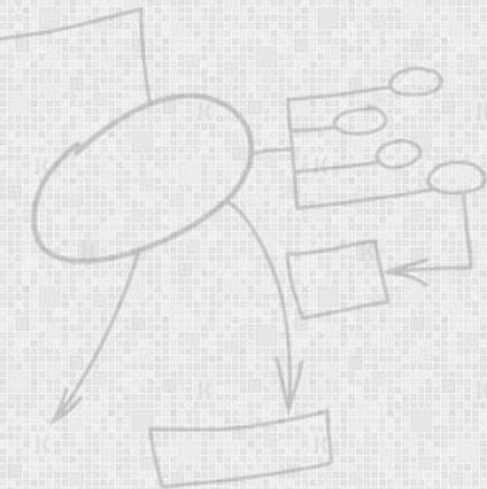


Configuration Topology

- A set of specific **associations between the components and connectors** of a software system's architecture.



Architectural Patterns



Layered Pattern

- **Context**

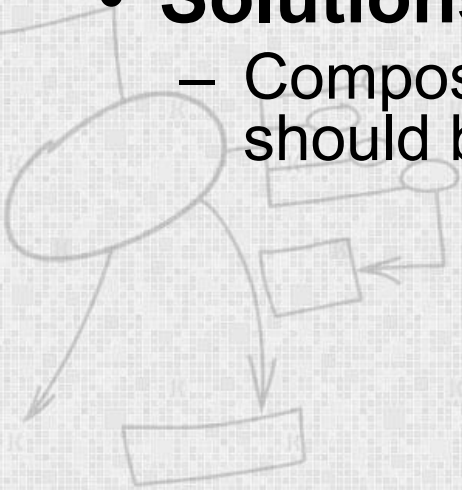
- You are working with a large, complex system and you want to manage complexity by decomposition.

- **Problem**

- How do you structure an application to support such operational requirements as maintainability, scalability, extensibility, robustness, and security?

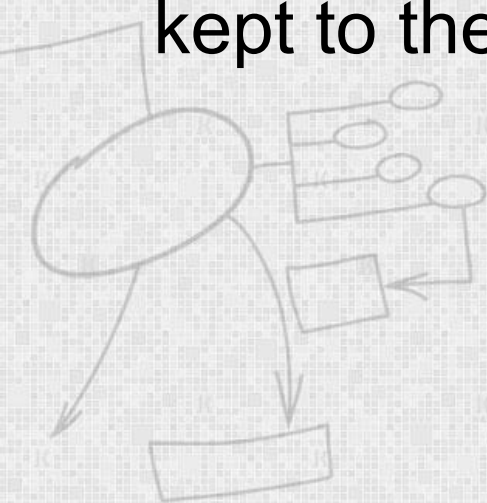
- **Solutions**

- Compose the solution into a set of layers. Each layer should be cohesive



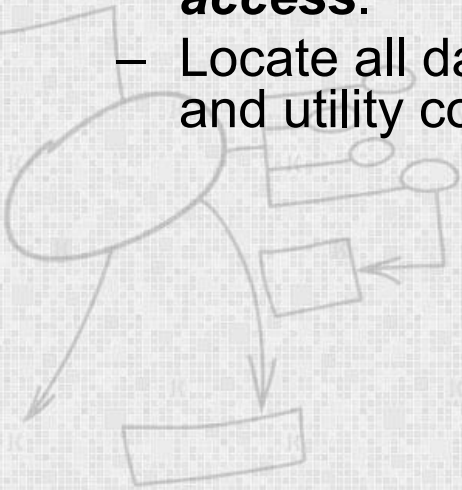
Layered Pattern

- Layering consists of a ***hierarchy of layers***, each ***providing service to the layer above*** it and ***serving as client to the layer below***.
- Interactions among layers are defined by suitable communication protocols.
- Interactions among non-adjacent layers must be kept to the minimum possible.

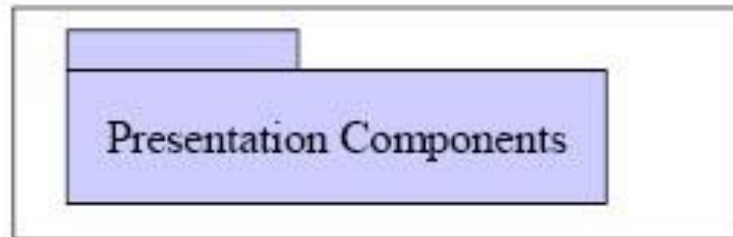


Three-Layered Pattern

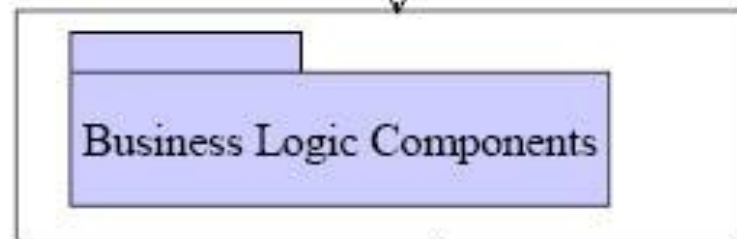
- **Context**
 - You are building a business solution using layers to organize your application (Information System)
- **Problem**
 - How do you organize your application to reuse business logic, provide deployment flexibility and conserve valuable resource connections?
- **Solutions**
 - Create three layers: ***presentation, business logic*** and ***data access***.
 - Locate all database-related code, including database clients access and utility components, in the data access layer.



Presentation Layer



Domain Layer

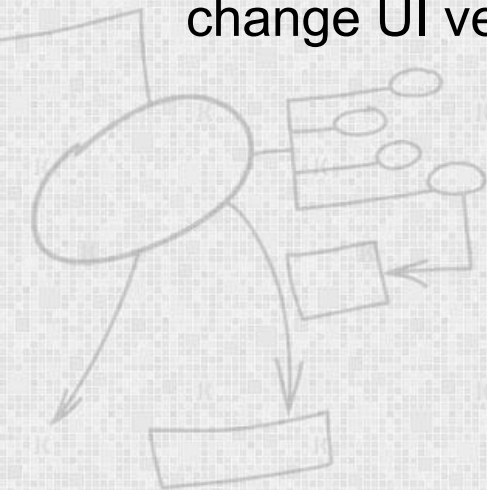


Data Access Layer



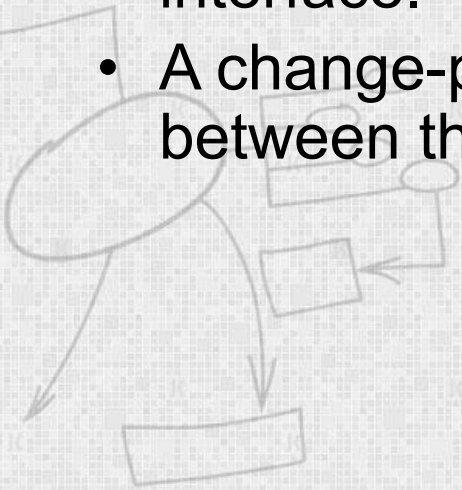
Why 3-Tier Architecture?

- In our first windows based PoS app, we coder people started to write all codes in behind of windows form. Problem started after few days. How? When our manager announced that we have to develop one web version of same product. Again we started to develop web pages from scratch and started to write all code. Totally mess.
- So, this is the problem and the solution is tired architecture. If we separate our codes by layer then changes in one layer will not affect much to another one. Tomorrow if business demands, we can change UI very quickly by using existing code.



Model-View-Controller

- The Model-View-Controller architectural pattern (MVC) divides an interactive application into three components.
 - The model contains the core functionality and data.
 - Views display information to the user.
 - Controllers control the view and handle user input.
- Views and controllers together comprise the user interface.
- A change-propagation mechanism ensures consistency between the user interface and the model.



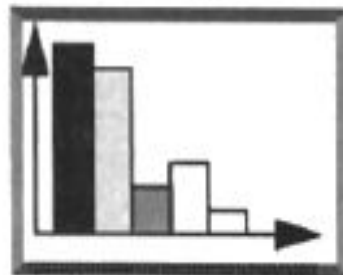
Example

core data

Black: 43%
Red: 39%
Blue: 6%
Green: 10%
Others: 2%



pie chart



bar chart



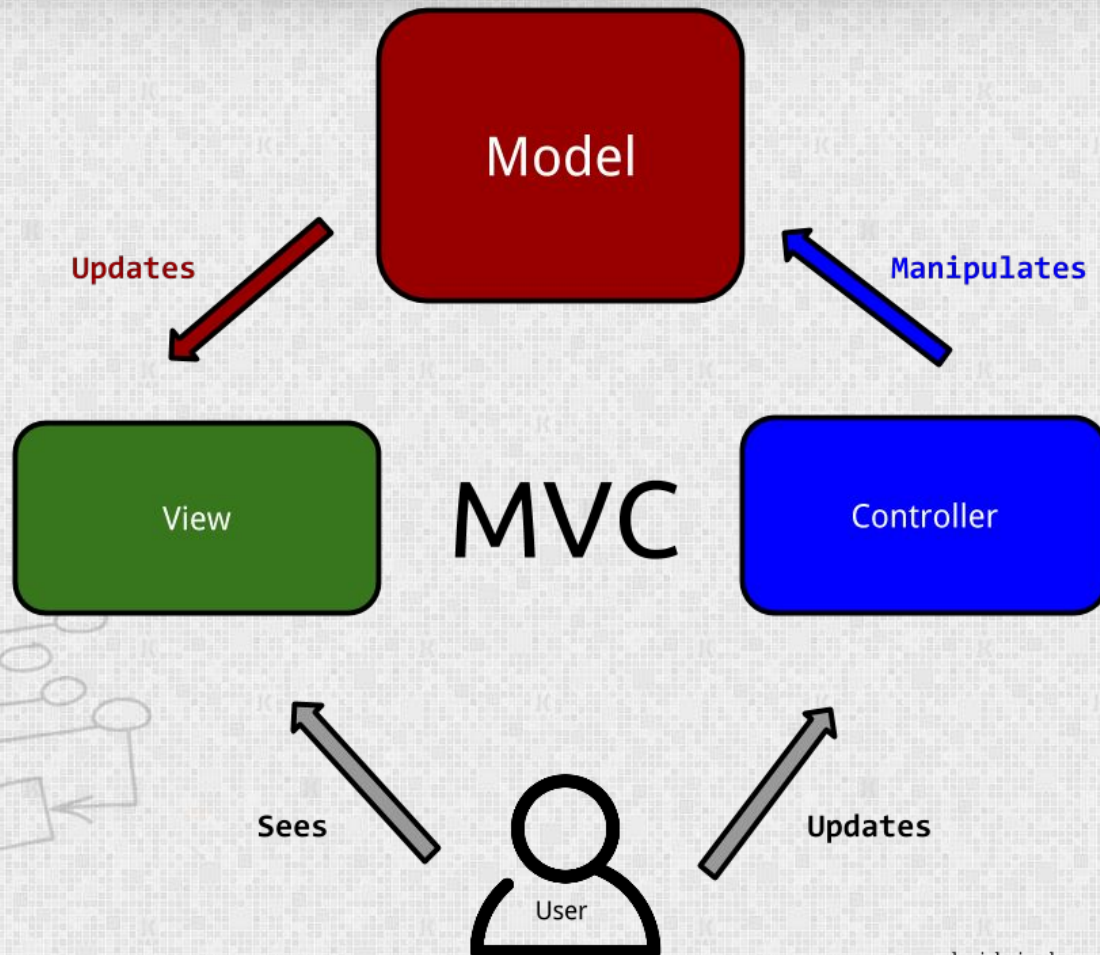
parliament

Black	43
Red	39
Blue	6
Green	10
Others	2

spreadsheet

Context: Interactive applications with a flexible human-computer interface.

MVC Architecture



danielmiessler.com



Problem

- The same information is presented differently in different windows, for example, in a bar or pie chart.
- The display and behavior of the application must reflect data manipulations immediately.
- Supporting different 'look and feel' standards or porting the user interface should not affect code in the core of the application.



Solution

- MVC divides an interactive application into the three areas: ***processing, output, and input.***
- ***The model component encapsulates core data and functionality.*** The model is independent of specific output representations or input behavior.
- ***View components display information to the user.*** A view obtains the data from the model. There can be multiple views of the model.
- ***Each view has an associated controller component.*** Controllers receive input, usually as events that encode mouse movement, activation of mouse buttons, or keyboard input. Events are translated to service requests for the model or the view. The user interacts with the system solely through controllers.

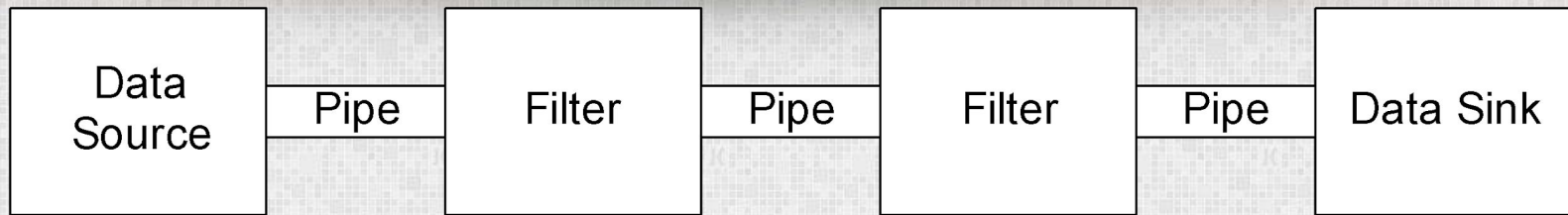


Pipes & Filters

- You are designing a system that needs to perform several transformations on some data
- The transformations can be performed sequentially
- It should be easy to:
 - Reorder the sequence of the transformations
 - Add new kinds of transformations
 - Remove transformations
- There are different sources of input data (files, network, programs)
- There are different destinations for output data (files, network, programs)



Solution



- Organize the system using Pipes & Filters
- Input data originates from Data Source components
- Output data is consumed by Data Sink components
- Each transformation required to convert the input into the output is implemented by a Filter component
- Data Sources, Filters, and Data Sinks arranged in a pipeline
- Pipes connect adjacent components, the output of one component becoming the input to the next component

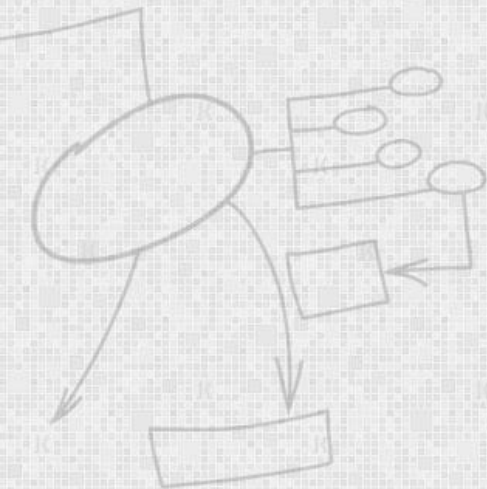


Implementation

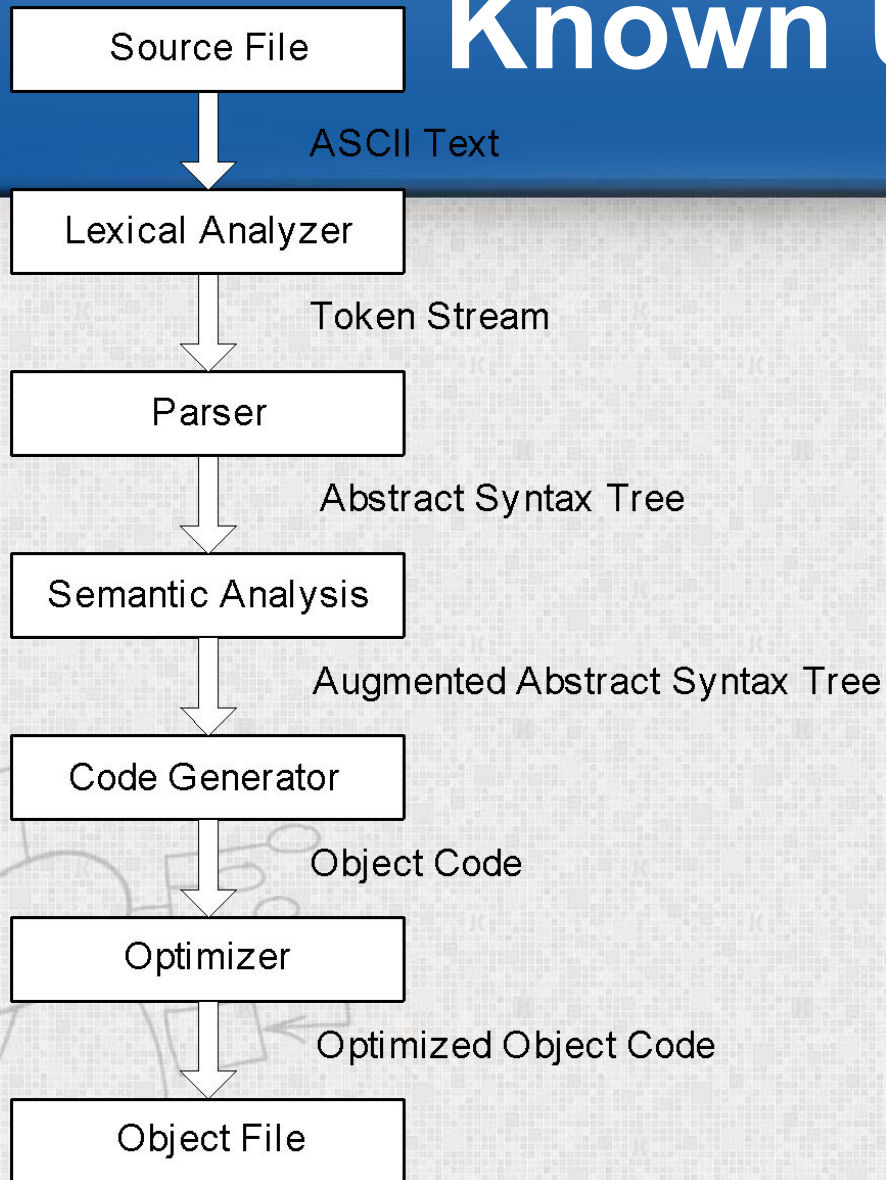
- Divide the system into a sequence of processing stages
- Define the data format to be passed along each pipe
- Decide how to implement each pipe connection
 - The simplest Pipe is direct Read/Write method calls between adjacent filters
 - If buffering or synchronization is required between filters, actual Pipe objects will be needed to perform these functions



Known Uses: Image Processing

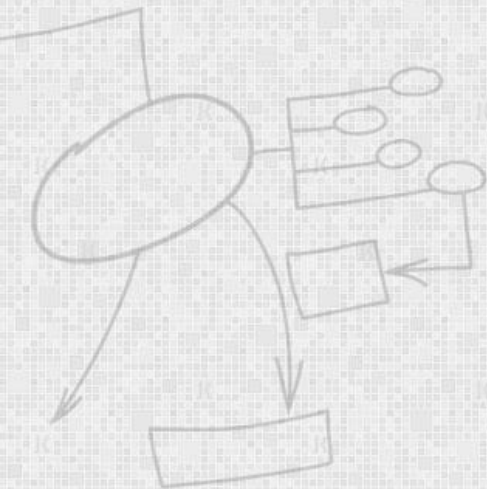


Known Uses: Compilers



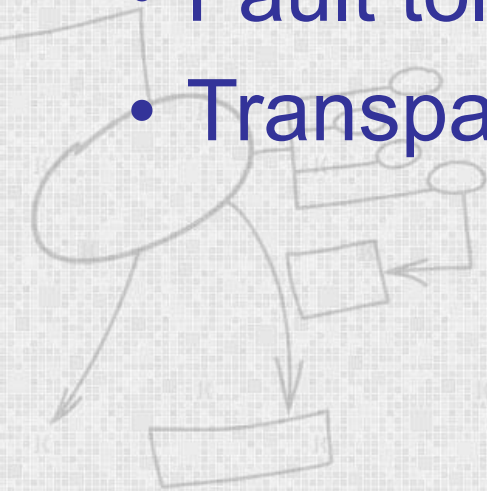
Distributed Systems Architectures

- Architectural design for software that executes on more than one processor



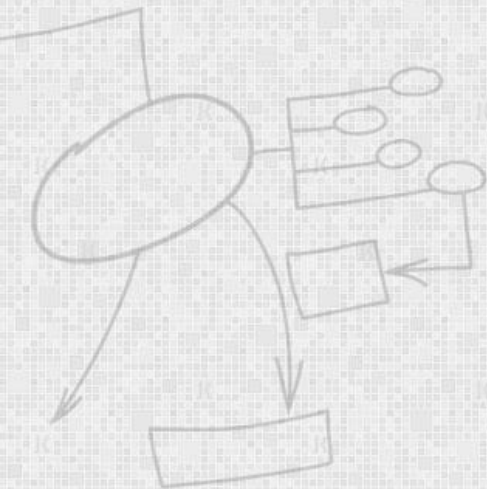
Distributed system characteristics

- Resource sharing
- Openness
- Concurrency
- Scalability
- Fault tolerance
- Transparency



Distributed system issues

- Complexity
- Security
- Manageability



Distributed systems architectures

- **Client-server architectures**

- Distributed services which are called on by clients. Servers that provide services are treated differently from clients that use services

- **Distributed object architectures**

- No distinction between clients and servers. Any object on the system may provide and use services from other objects

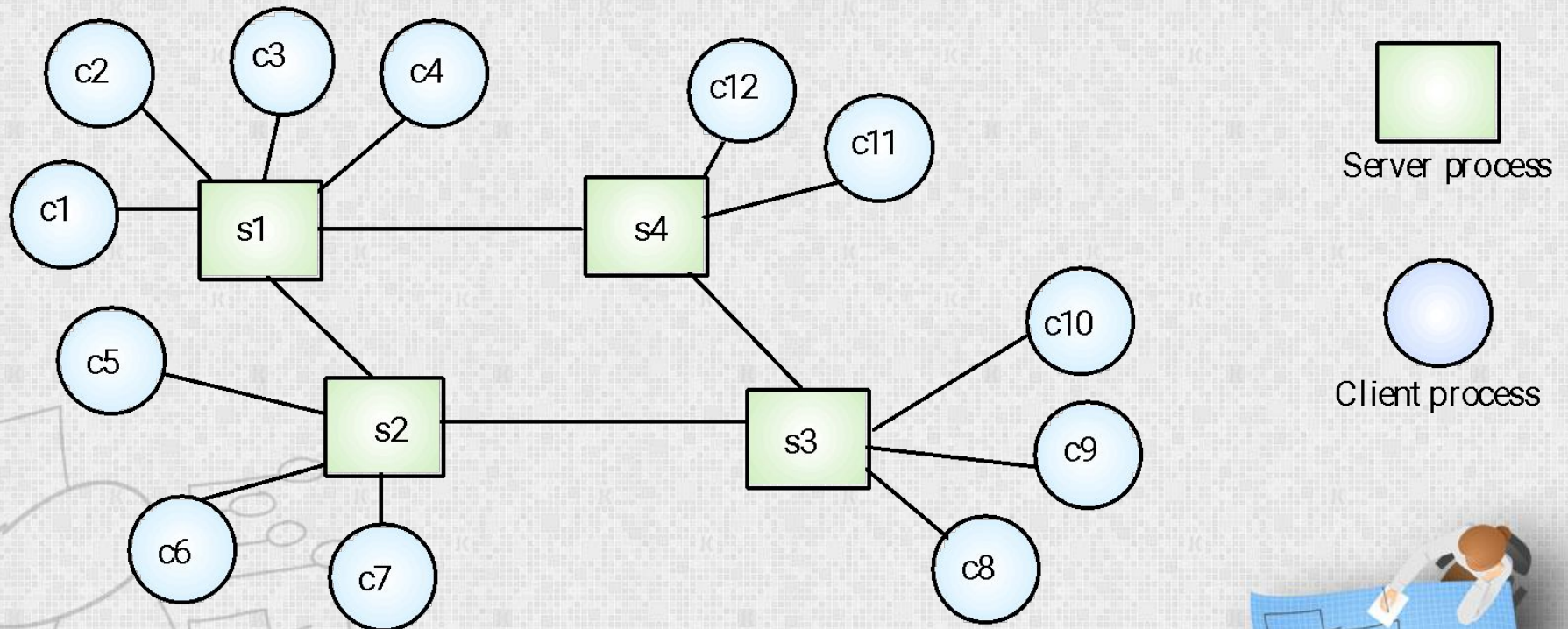


Client-server architectures

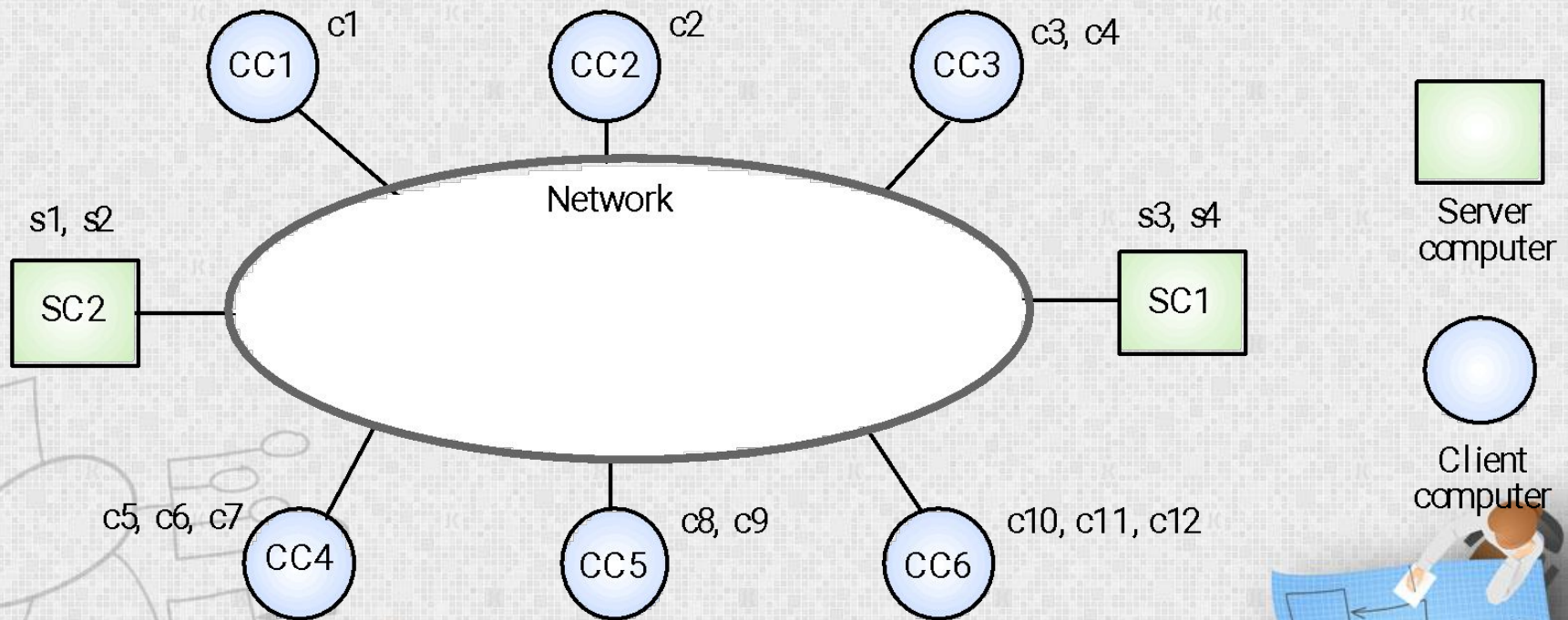
- **The application is modelled as a set of services** that are provided by servers and a set of clients that use these services
- **Clients know of servers but servers need not know of clients**
- Clients and servers are logical processes
- The mapping of processors to processes is not necessarily 1 : 1



A client-server system



Computers in a C/S network



Thin and fat clients

- ***Thin-client model***

- In a thin-client model, all of the application processing and data management is carried out on the server. The client is simply responsible for running the presentation software.

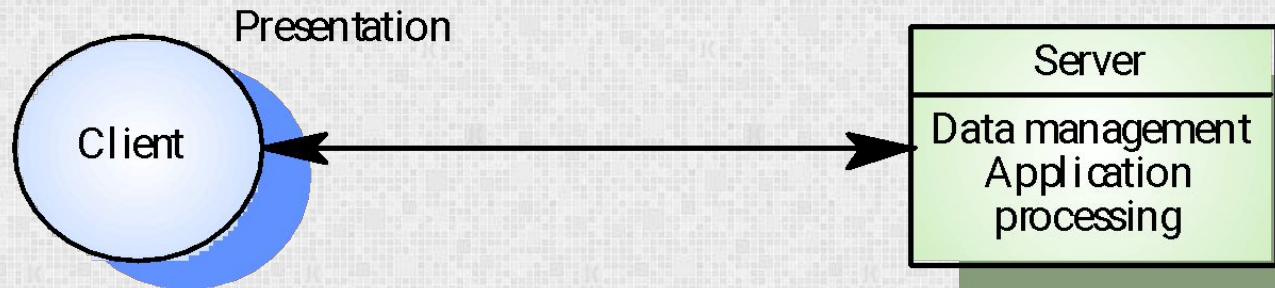
- ***Fat-client model***

- In this model, the server is only responsible for data management. The software on the client implements the application logic and the interactions with the system user.

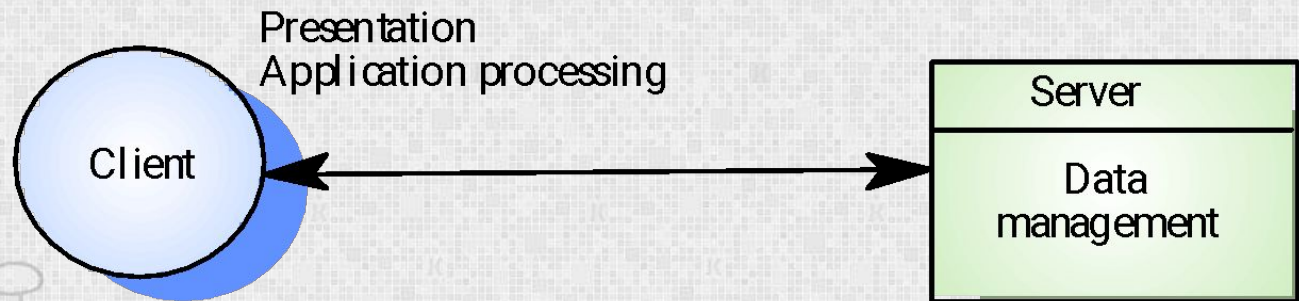


Thin and fat clients

Thin-client model

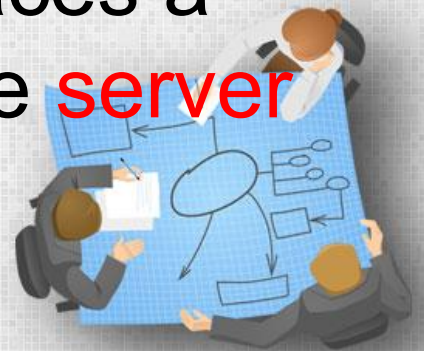


Fat-client model



Thin client model

- Normally used when legacy systems are migrated to client server architectures.
 - The legacy system acts as a server in its own right with a graphical interface implemented on a client
- A major disadvantage is that it places a heavy processing load on both the server and the network
- Cloud Computing



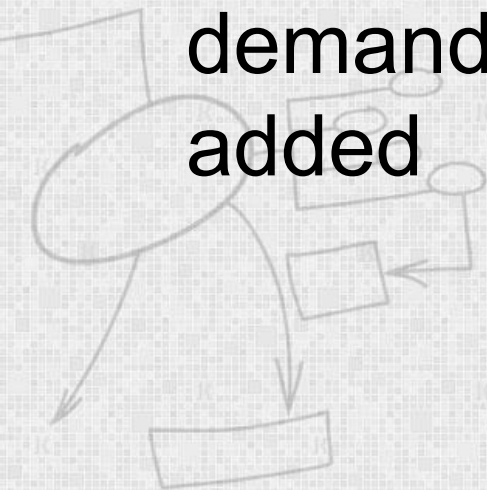
Fat client model

- More processing is delegated to the client as the application processing is locally executed
- Most suitable for new C/S systems where the capabilities of the client system are known in advance
- More complex than a thin client model especially for management. New versions of the application have to be installed on all clients

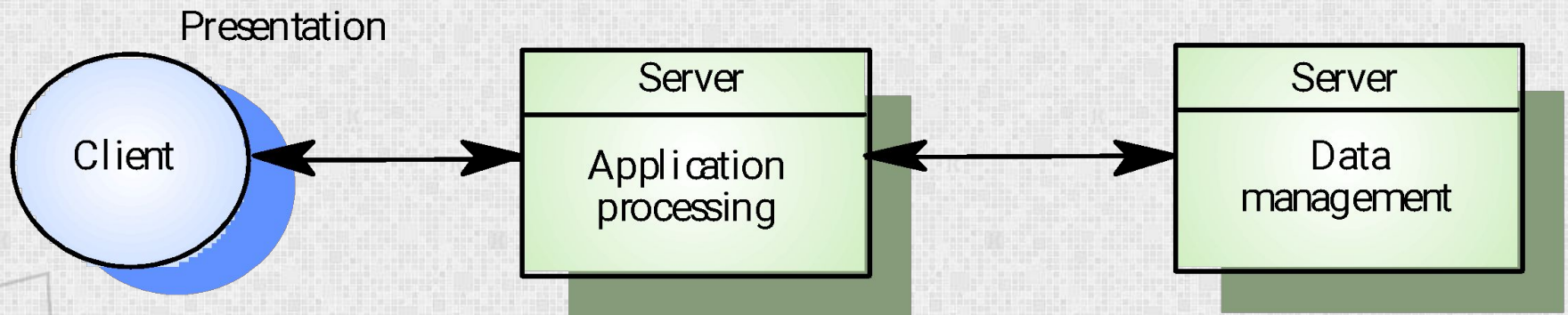


Three-tier C/S

- In a three-tier client server architecture, each of the application architecture layers may execute on a separate processor
- A more scalable architecture - as demands increase, extra servers can be added



A 3-tier C/S architecture

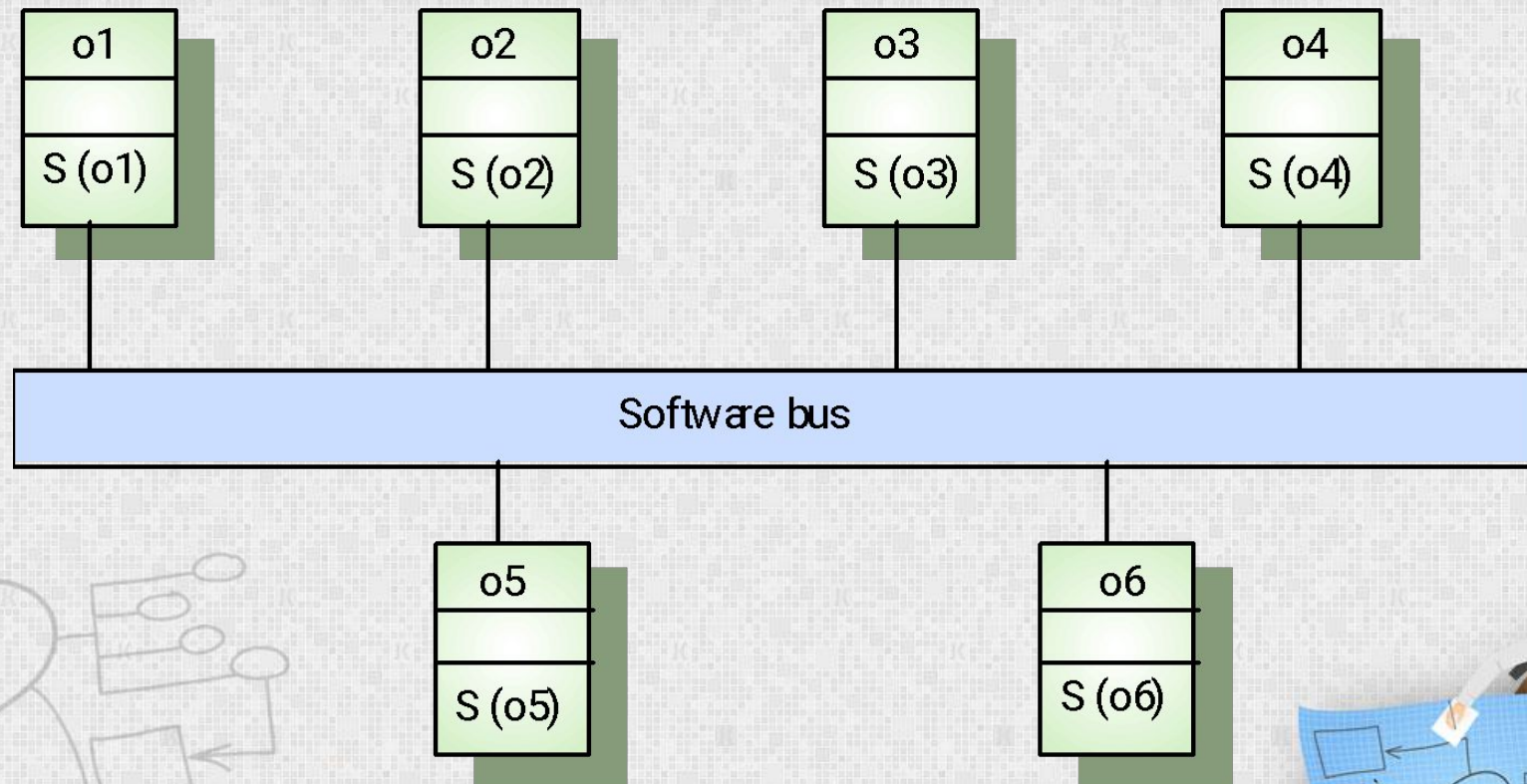


Distributed object architectures

- **There is no distinction in a distributed object architectures between clients and servers**
- Each distributable entity is an object that
 - provides services to other objects and
 - receives services from other objects
- Object communication is through a middleware system called an object request broker (software bus)
- However, more complex to design than C/S systems

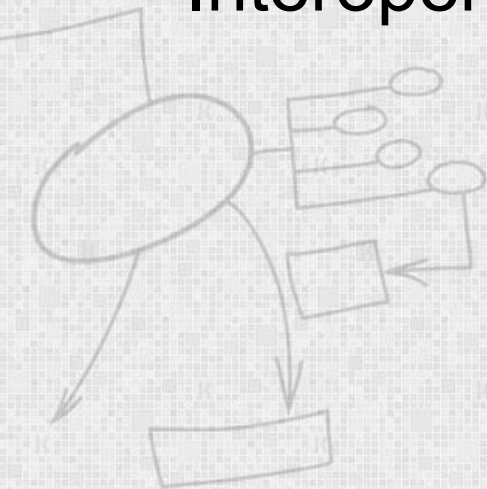


Distributed object architecture

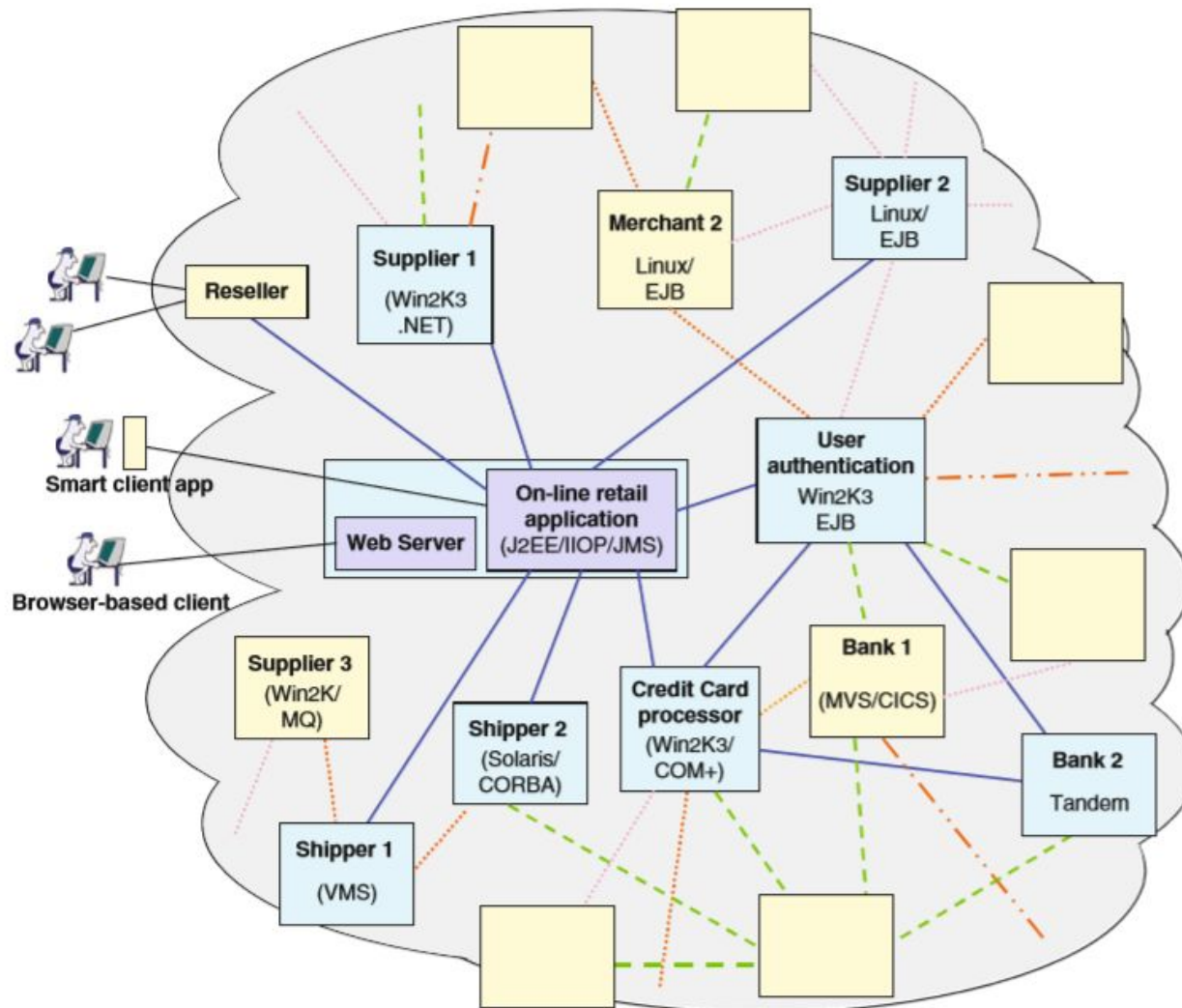


Service Oriented Architectures

- Latest step in the development of application integration middleware.
- They attempt to fix the interoperability problems of the past and provide a foundation for future Internet-scale distributed applications.
- Interoperability?

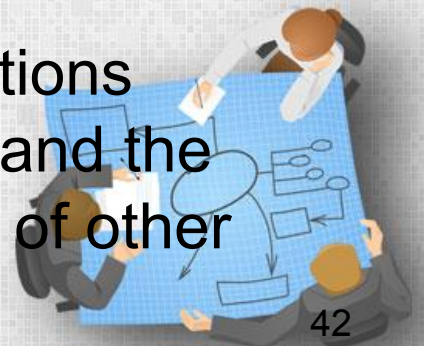


Service Oriented Architectures



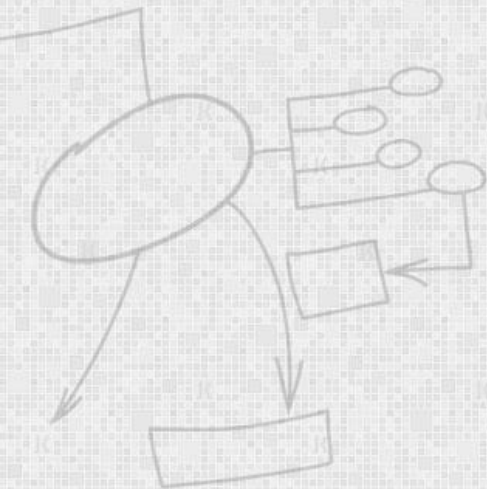
Web Services

- A **set** of integration technology **standards** that were designed specifically to meet the **requirements** arising from **service-oriented architectures** and systems.
- Web services, provide four basic functions that let developers (and programs) do the following:
 - Find suitable services (using UDDI or another directory)
 - Find out about a service (using WSDL)
 - Ask a service to do something (using SOAP)
 - Make use of services
- Services are defined using XML, and applications request services by sending XML messages and the Web services standards make extensive use of other existing XML standards wherever possible.



SOAP

- Simple Object Access Protocol
- It specifies a simple but extensible XML-based application-to application communication protocol.



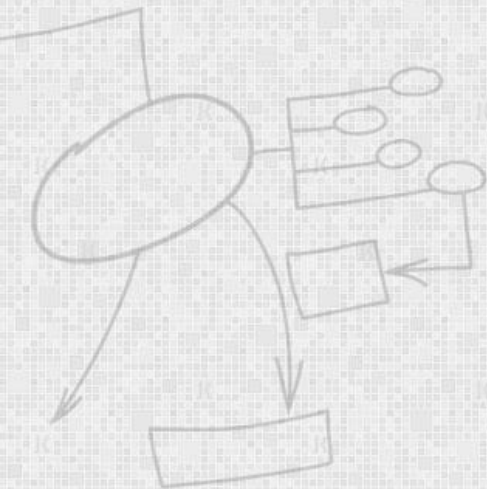
UDDI and WSDL

- SOAP services are normally described using WSDL (Web Services Description Language) including their interfaces, methods, and parameters and can be located by searching a UDDI (Universal Description, Discovery, and Integration) directory.



Web Services Definition

- There is also increasingly good tool and library / framework support for these standards, so developers only have to understand the capabilities offered rather than the detailed XML syntax



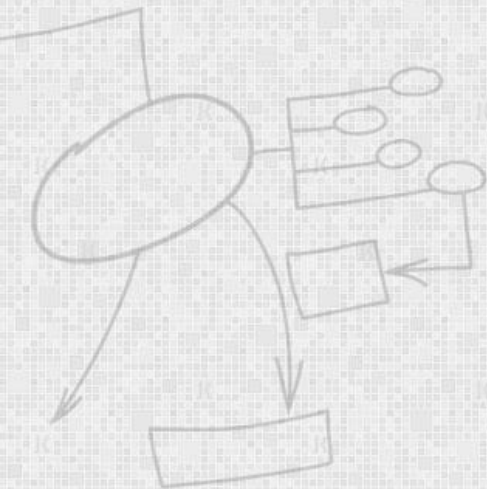
Web Services Definition

```
package brokerservice.endpoint;  
import javax.jws.WebService;  
@WebService  
public class Broker {  
    @WebMethod  
    public String viewStock(String name)  
    {  
        // code omitted  
    }  
}
```



Web Services Definition

- So, with toolkits like JAX-WS, the service developer does not need to create or understand XML messages formatted as SOAP.
- The JAX-WS run-time system simply converts the API calls and responses to and from underlying SOAP message formats.



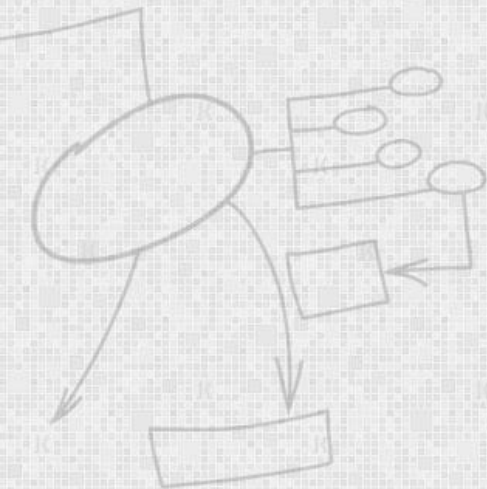
.Net Example

```
public class WebService1 : System.Web.Services.WebService
{
    [WebMethod]
    public string HelloWorld()
    {
        return "Hello World";
    }
}
```

```
<?xml version="1.0" encoding="UTF-8"?>
- <wsdl:definitions xmlns:wsdl="http://schemas.xmlsoap.org/wsdl/" targetNamespace="http://tempuri.org/" xmlns:http="http://schemas.xmlsoap.org/wsdl/http/"
  xmlns:soap12="http://schemas.xmlsoap.org/wsdl/soap12/" xmlns:s="http://www.w3.org/2001/XMLSchema"
  xmlns:soap="http://schemas.xmlsoap.org/wsdl/soap/" xmlns:tns="http://tempuri.org/" xmlns:mime="http://schemas.xmlsoap.org/wsdl/mime/"
  xmlns:soapenc="http://schemas.xmlsoap.org/soap/encoding/" xmlns:tm="http://microsoft.com/wsdl/mime/textMatching/">
  - <wsdl:types>
    - <s:schema targetNamespace="http://tempuri.org/" elementFormDefault="qualified">
      - <s:element name="HelloWorld">
        <s:complexType/>
      </s:element>
      - <s:element name="HelloWorldResponse">
        <s:complexType>
          - <s:sequence>
            <s:element name="HelloWorldResult" type="s:string" maxOccurs="1" minOccurs="0"/>
          </s:sequence>
        </s:complexType>
      </s:element>
    </s:schema>
  </wsdl:types>
  - <wsdl:message name="HelloWorldSoapIn">
    <wsdl:part name="parameters" element="tns:HelloWorld"/>
  </wsdl:message>
  - <wsdl:message name="HelloWorldSoapOut">
    <wsdl:part name="parameters" element="tns:HelloWorldResponse"/>
  </wsdl:message>
  - <wsdl:portType name="WebService1Soap">
    - <wsdl:operation name="HelloWorld">
      <wsdl:input message="tns:HelloWorldSoapIn"/>
      <wsdl:output message="tns:HelloWorldSoapOut"/>
    </wsdl:operation>
  </wsdl:portType>
  - <wsdl:binding name="WebService1Soap" type="tns:WebService1Soap">
    <soap:binding transport="http://schemas.xmlsoap.org/soap/http"/>
    - <wsdl:operation name="HelloWorld">
```

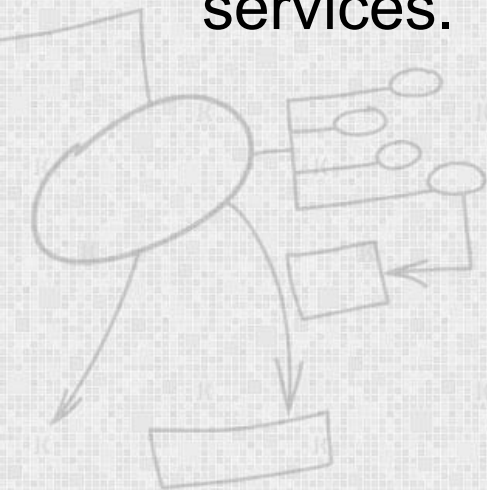

eXtensible Markup Language

- Markup Language
- Meta Language
- XML-based documents are generally used for describing and exchanging data



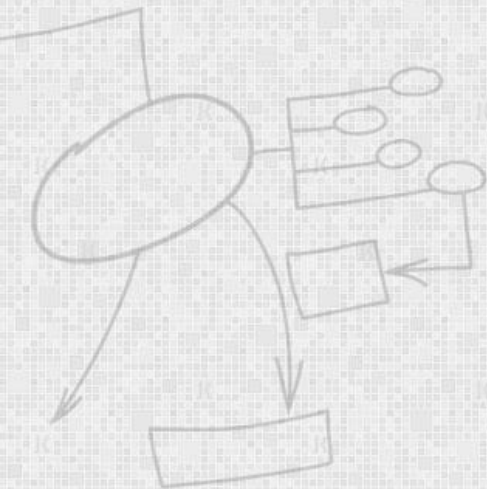
RESTful Web Services

- **RESTful** Web services rely on **HTTP** as a sufficiently rich protocol to completely meet the needs of Web services applications.
- In the REST model, the HTTP **GET, POST, PUT**, and **DELETE** verbs are used to transfer data (often in the form of XML/JSON documents) between client and services.



RESTful Web Services Example

- Example: A simple customer database Web service using a RESTful approach.
- In this example, the customer database is a “resource” and the individual customer records are also “resources”
- Each of these resources has a unique URI that can be used as the subject of an HTTP verb.



RESTful Web Services

- The URI <http://example.com/customers> identifies the customer database resource. GET requests sent to this URI return the set of all customers as a single XML/JSON document containing a list of URIs that point to the individual customer resources.
- The URI for each customer in the database is formed by appending the customer's unique ID to the customer set URI. For example, <http://example.com/customers/1> identifies the resource corresponding to the customer with ID 1.



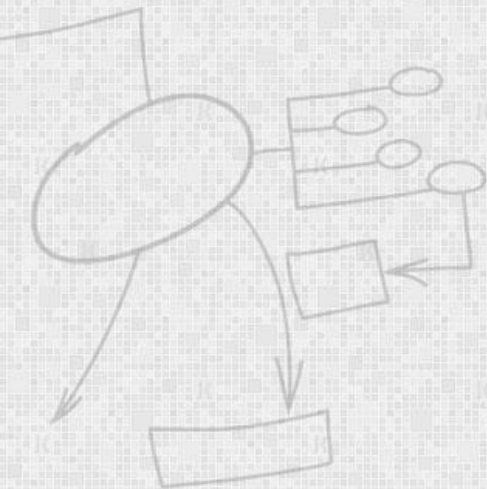
RESTful Web Services

- A **GET** request sent to one of these unique customer URIs retrieves an XML document that contains a representation of the current state of the corresponding customer.
- Existing customer resources can be **updated** by **PUTting** an XML document containing a representation of the desired new state of the customer to the appropriate customer URI.



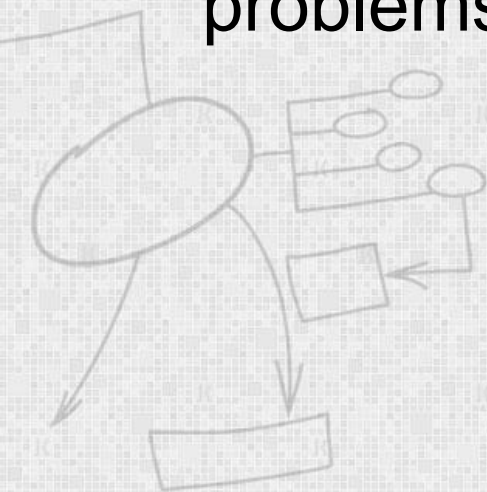
RESTful Web Services

- **New customers** can be added to the database by **POSTing** XML documents containing the representation of the new resource to the collection URI.
- Customer entries can be **deleted** by sending a **DELETE** request to the customer URI.



RESTful Web Services

- Many of the arguments of the RESTful advocates come from their belief that REST is “**simple**” and SOAP are “**complex**”.
- Of course, “simplicity” and “complexity” are **relative** to the architectural and technical problems you are trying to solve



RESTful Web Services

- The choice is yours as SOAP and REST are really complementary approaches to implementing Web services, each best suited to different kinds of distributed applications.

